Lab 4: Weather Stations Monitoring

Final Report

PART A — Weather Station Mock

A.1 Objective

The objective of Part A is to implement a weather station mock that generates weather-status messages periodically. Each message contains the station ID, sequence number, battery status, timestamp, and weather information. The station also simulates message loss and generates random battery and weather values according to the required specifications.

A.2 Weather.java

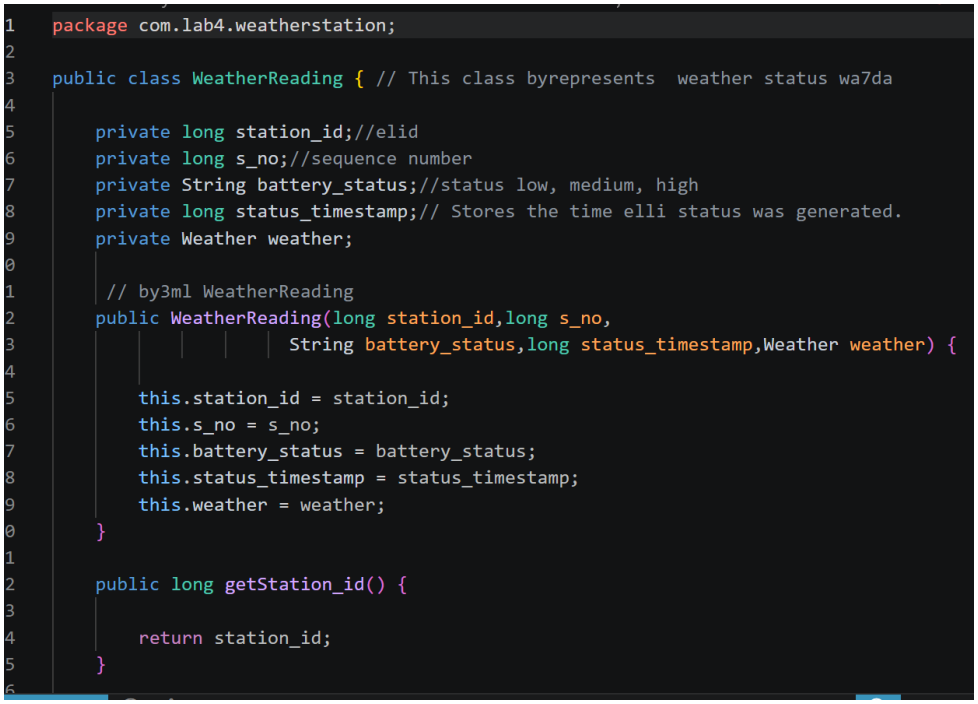
The Weather class represents the weather information contained inside each weather-status message. It stores humidity, temperature, and wind speed.

```java
package com.lab4.weatherstation;
public class Weather {
    private int humidity;
    private int temperature;
    private int wind_speed;

    public Weather(int humidity, int temperature, int wind_speed) {
        this.humidity = humidity;

        this.temperature = temperature;

        this.wind_speed = wind_speed;
    }

    public int getHumidity() {
        return humidity;
    }

    public int getTemperature() {
        return temperature;
    }

    public int getWind_speed() {
        return wind_speed;
    }
}
```

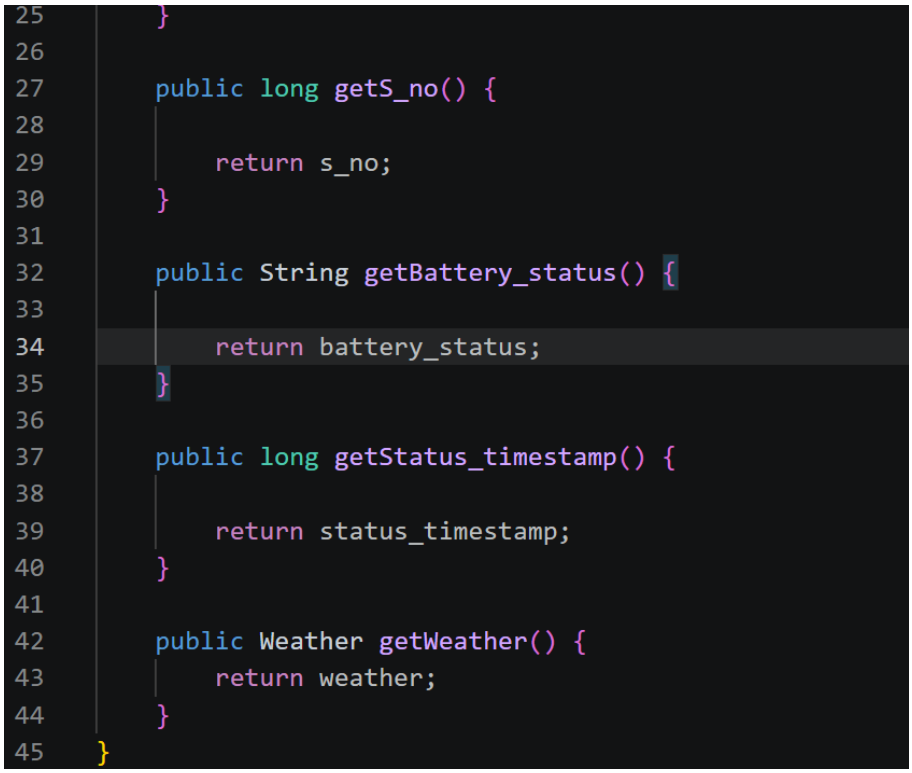
Figure A1: Complete implementation of the Weather class. The class stores the three weather parameters and provides getter methods to access them.

A.3 WeatherReading.java

The WeatherReading class represents one complete weather-status message. It contains the station ID, sequence number, battery status, timestamp, and the nested Weather object.



```java
package com.lab4.weatherstation;

public class WeatherReading { // This class byrepresents  weather status wa7da

    private long station_id;//elid
    private long s_no;//sequence number
    private String battery_status;//status low, medium, high
    private long status_timestamp;// Stores the time elli status was generated.
    private Weather weather;

     // by3ml WeatherReading
    public WeatherReading(long station_id,long s_no,
                          String battery_status,long status_timestamp,Weather weather) {

        this.station_id = station_id;
        this.s_no = s_no;
        this.battery_status = battery_status;
        this.status_timestamp = status_timestamp;
        this.weather = weather;
    }

    public long getStation_id() {

        return station_id;
    }
```



```java
    }

    public long getS_no() {

        return s_no;
    }

    public String getBattery_status() {

        return battery_status;
    }

    public long getStatus_timestamp() {

        return status_timestamp;
    }

    public Weather getWeather() {
        return weather;
    }
}
```

Figure A2: Complete implementation of the WeatherReading class. The Weather object is nested inside the reading so that the generated JSON follows the required structure.

A.4 WeatherStation.java

The WeatherStation class is responsible for generating the weather readings. It generates random weather values, assigns sequence numbers and timestamps, simulates message loss, creates the WeatherReading object, converts it to JSON using Jackson, and outputs the generated message.

Ubuntu > home > maryam > netcentric-lab4 > weather-station > src > main > java > com > lab4 > weatherstation > J WeatherStation.java

```java
package com.lab4.weatherstation;//by3ml weather msgs
import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.kafka.clients.producer.*;//3ashan ab3at l kafka
import org.apache.kafka.common.serialization.StringSerializer;//en el values strings
import java.util.Random;
import java.util.Properties;

public class WeatherStation {

    private static final long STATION_ID = 1;//prvate bass lelclass dah.static brlongs lelclass
    private static final Random random = new Random();
    private static final ObjectMapper objectMapper = new ObjectMapper();//by3ml convertor

    public static void main(String[] args) throws Exception {

        Properties properties = new Properties();//bya3mel collection of kafka configrations

        properties.setProperty(
                ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
                "kafka:9092"//connect lelkafka felmakan dah
        );

        properties.setProperty(
                ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
                StringSerializer.class.getName()//men string l bytes
        );
        properties.setProperty(
                ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
                StringSerializer.class.getName()
        );

        KafkaProducer<String, String> producer = new KafkaProducer<>(properties);//elkey wel value

        long sequenceNumber = 1;

        while (true) {

            // Randomly drop 10% of messages
            if (random.nextDouble() >= 0.10) {

                String batteryStatus = generateBatteryStatus();
                int humidity = 20 + random.nextInt(71);//bein 20 w 90%
                int temperature = 10 + random.nextInt(31);//bein 10 w 40
                int windSpeed = random.nextInt(21);//men 0 l 20
                long status_timestamp = System.currentTimeMillis() / 1000;//akhaleeeha sec
                Weather weather = new Weather(
                        humidity,
                        temperature,
                        windSpeed
```

```
            WeatherReading reading = new WeatherReading(
                STATION_ID,
                sequenceNumber,
                batteryStatus,
                status_timestamp,
                weather
            );

            String json = objectMapper.writeValueAsString(reading);

            System.out.println(json);

            ProducerRecord<String, String> record =
                    new ProducerRecord<>("weather-readings", json);//message feeha eltopic w elvalue

            producer.send(record);
        }

        sequenceNumber++;

        Thread.sleep(1000);//astana sania
```

```
    private static String generateBatteryStatus() {

        double value = random.nextDouble();

        if (value < 0.30) {
            return "low";
        } else if (value < 0.70) {
            return "medium";
        } else {
            return "high";
        }
    }
}
```

Figure A3: Complete implementation of the weather station mock. The program generates readings approximately every second, randomly drops approximately 10% of messages, generates the required weather and battery values, and converts each reading into JSON.

A.5 Part A Compilation Test

The Part A implementation was compiled using Maven to verify that all Java classes and their dependencies were correctly configured.

```
maryam@Maryooma:~/netcentric-lab4$ cd ~/netcentric-lab4/weather-station
mvn clean compile
```

```
[INFO] BUILD SUCCESS
[INFO] ------------------------
```

Figure A4: Successful compilation of the Part A weather-station application using Maven.

A.6 Part A Runtime Test

The weather station was then executed to verify that it generates weather-status messages in the required JSON format.

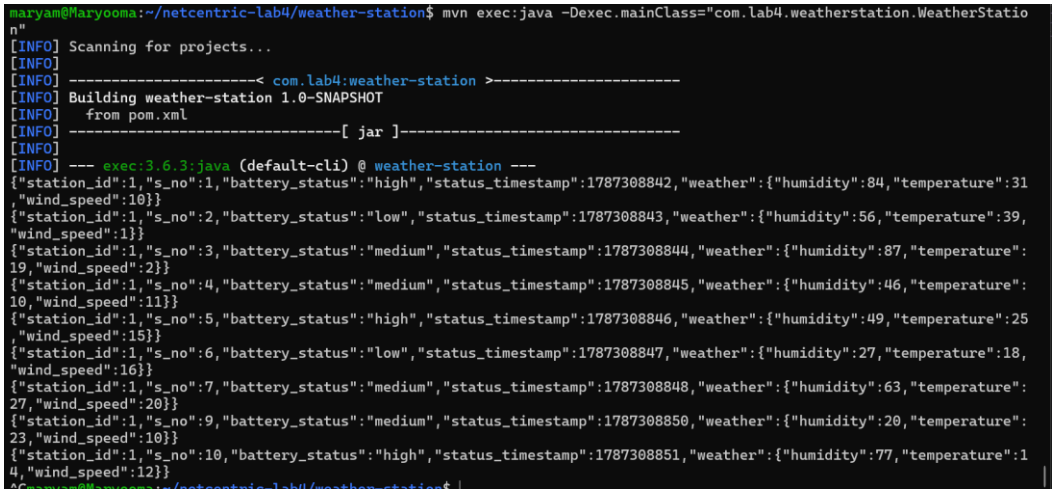


Figure A5: Runtime output showing the generated weather-status messages in JSON format.

A.7 Message Drop Test

The station simulates unreliable communication by randomly dropping approximately 10% of attempted messages. Since the sequence number is incremented even when a message is dropped, missing sequence numbers provide evidence of dropped messages.

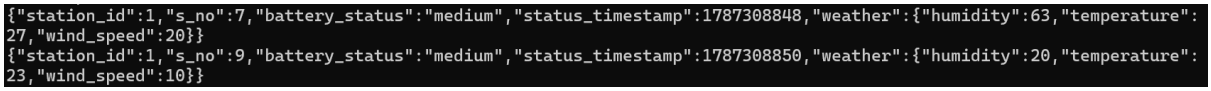


Figure A6: Missing sequence number demonstrating the random message-drop mechanism.

A.8 Statistical Validation

A larger statistical test was performed to verify the approximately 10% message-drop rate and the required battery-status distribution of approximately 30% low, 40% medium, and 30% high.

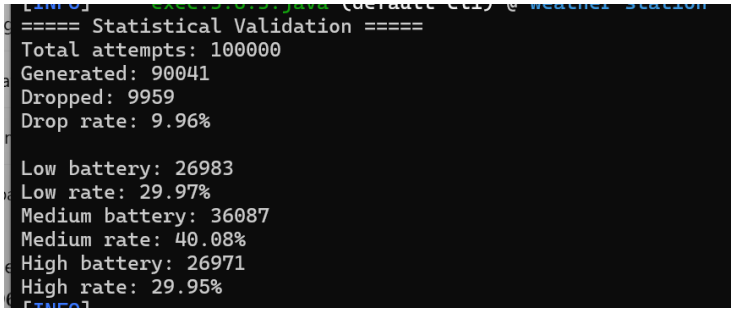


Figure A7: Statistical validation of 100,000 message attempts. The results show a 9.96% message-drop rate, which is close to the required 10%. The battery-status distribution was also approximately 30% low, 40% medium, and 30% high, satisfying the required distribution.

Requirement	Expected	Your result	Status
Message drop	≈ 10%	**9.96%**	PASS
Low battery	≈ 30%	**29.97%**	PASS
Medium battery	≈ 40%	**40.08%**	PASS
High battery	≈ 30%	**29.95%**	PASS

A.9 Part A Conclusion

Part A was successfully implemented and tested. The weather station generates the required weather-status structure, assigns sequence numbers and timestamps, generates the required weather and battery values, simulates message loss, and produces valid JSON messages.

PART B — Kafka Producer

B.1 Objective

The objective of Part B is to connect the weather station to Apache Kafka using the Java Kafka Producer API. The generated JSON weather readings are published to the weather-readings Kafka topic.

B.2 Kafka and ZooKeeper Configuration

Kafka and ZooKeeper were deployed inside the Minikube Kubernetes cluster. Kafka is accessible internally using the Kubernetes service address kafka:9092

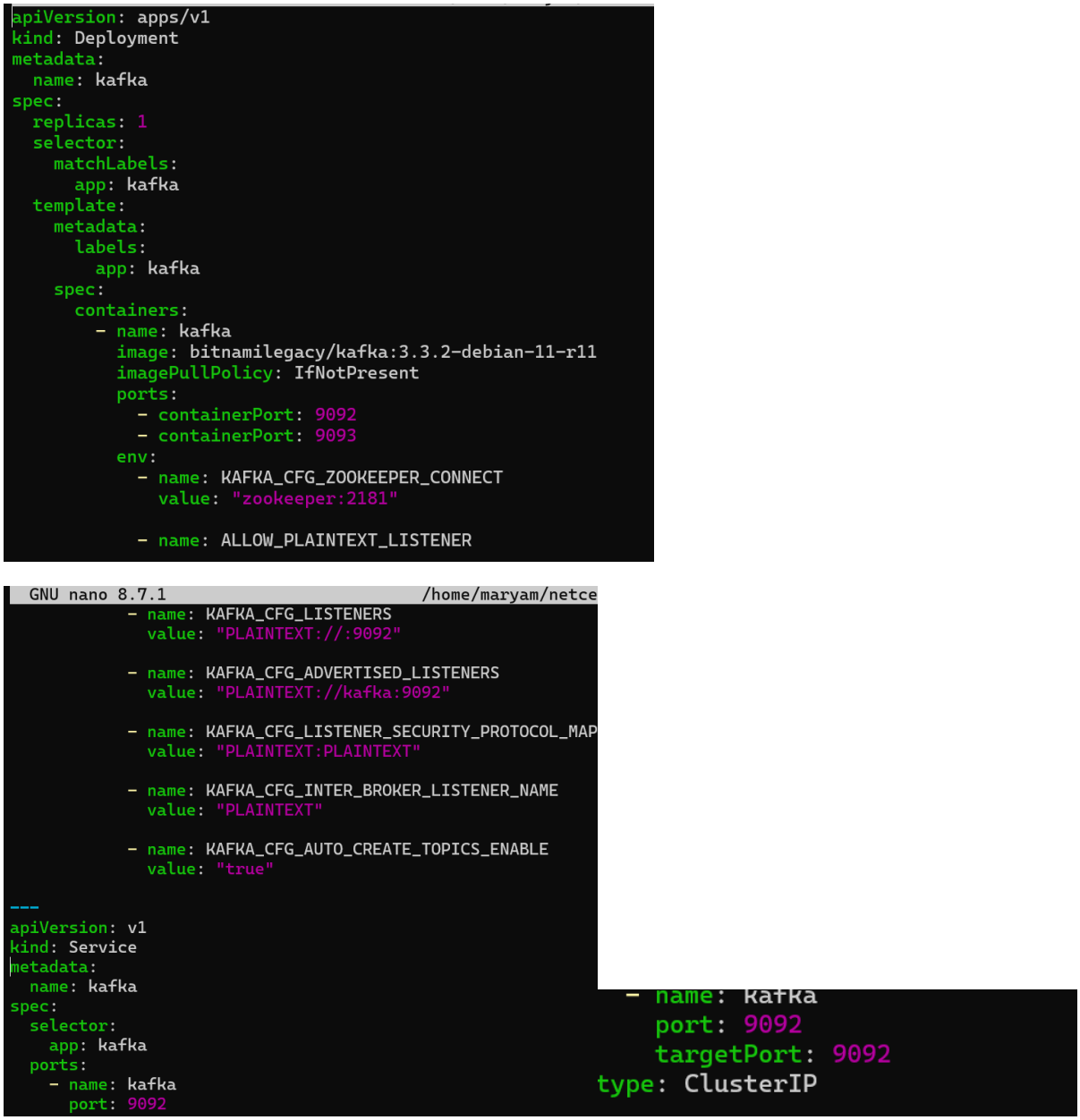


Figure B1: Kubernetes configuration used to deploy Kafka and expose it through the kafka service.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: zookeeper
spec:
  replicas: 1
  selector:
    matchLabels:
      app: zookeeper
  template:
    metadata:
      labels:
        app: zookeeper
    spec:
      containers:
        - name: zookeeper
          image: bitnamilegacy/zookeeper:3.9.3-debian-12-r22
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 2181
          env:
            - name: ALLOW_ANONYMOUS_LOGIN
              value: "yes"

---
apiVersion: v1
kind: Service
metadata:
  name: zookeeper
spec:
  selector:
    app: zookeeper
  ports:
    - port: 2181
      targetPort: 2181
  type: ClusterIP
```

Figure B2: Kubernetes configuration used to deploy ZooKeeper and expose it through the zookeeper service.

```
maryam@Maryooma:~/netcentric-lab4/weather-station$ kubectl get pods
kubectl get services
NAME                         READY   STATUS    RESTARTS   AGE
kafka-5c958ff9bc-74ght       1/1     Running   0          47h
mysql-6b884d6959-t58kl       1/1     Running   0          45h
zookeeper-5877c775d6-8jsfv   1/1     Running   0          47h
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
kafka        ClusterIP   10.108.6.173    <none>        9092/TCP   47h
kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP    47h
mysql        ClusterIP   10.97.176.227   <none>        3306/TCP   45h
zookeeper    ClusterIP   10.107.80.129   <none>        2181/TCP   47h
```

Figure B3: Kafka and ZooKeeper successfully running inside the Minikube cluster.

B.4 Kafka Producer Code.

The Part A weather station was extended with the Kafka Producer API. The producer is configured to connect to the Kafka broker and serialize both the message key and value as strings.

Complete Weather Station implementation with Kafka Producer functionality. The existing weather-generation logic is retained while the generated JSON messages are additionally published to Kafka.(full code in part A)

B.5 Publishing to Kafka

Each generated JSON message is encapsulated in a ProducerRecord and published to the weather-readings topic using the Kafka Producer API.

```
ProducerRecord<String, String> record =
        new ProducerRecord<>("weather-readings", json);//message feeha eltopic w

producer.send(record);
```

Figure B4: Creation and transmission of a Kafka record to the weather-readings topic.

B.6 Kafka Producer Test

The producer was tested by running the weather station and consuming messages from the weather-readings topic. The received messages were compared with the JSON generated by the weather station.

```
maryam@Maryooma:~/netcentric-lab4/weather-station$ kubectl exec -it deployment/kafka -- \
/opt/bitnami/kafka/bin/kafka-topics.sh \
--bootstrap-server kafka:9092 \
--describe \
--topic weather-readings
Topic: weather-readings TopicId: G0dNhM6bQMywBIDxAlLt8g PartitionCount: 1       ReplicationFactor: 1    Configs:
        Topic: weather-readings Partition: 0    Leader: 1001    Replicas: 1001  Isr: 1001
```

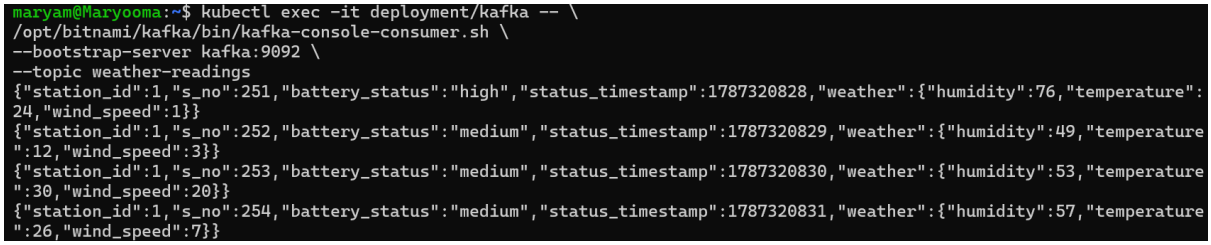


Figure B5: Kafka consumer successfully receiving JSON weather readings from the weather-readings topic.

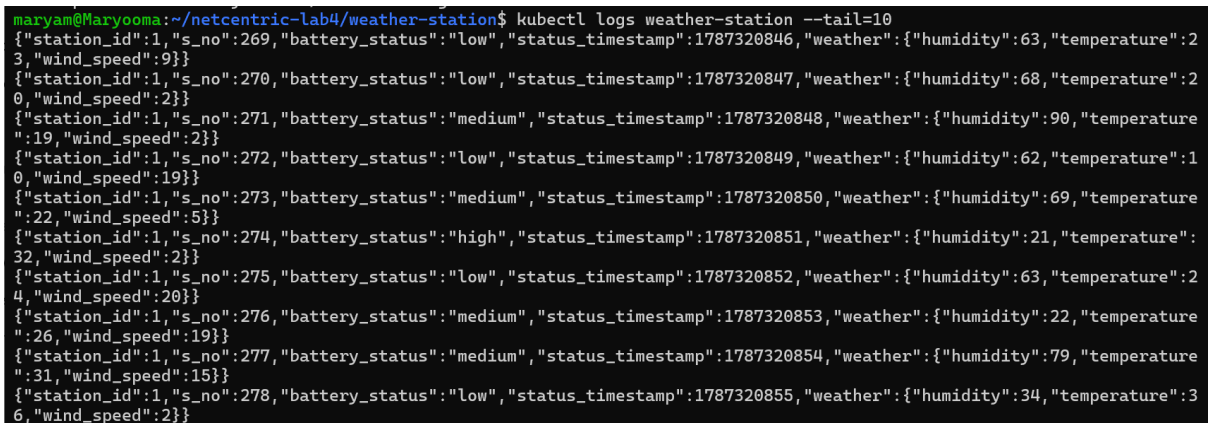


Figure B6: Weather Station Producer running inside the Minikube cluster and generating JSON weather readings.

B.7 Part B Conclusion

Part B was successfully implemented. The weather station successfully connects to Kafka and publishes the generated JSON weather readings to the weather-readings topic using the Java Kafka Producer API.

PART C — Raining Processor

C.1 Objective

The objective of Part C is to process the weather readings from Kafka and identify raining conditions. A reading is considered a raining condition when its humidity is greater than 70%.

C.2 Full RainingProcessor.java

A Kafka Streams application named RainingProcessor was implemented to consume the weather-readings topic, parse the JSON weather messages, extract the humidity value, and filter the messages according to the raining condition.

```java
package com.lab4.processor;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.KafkaStreams;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.StreamsConfig;
import org.apache.kafka.streams.kstream.KStream;
import java.util.Properties;

public class RainingProcessor {

    public static void main(String[] args) {

        Properties properties = new Properties();

        properties.setProperty(
                StreamsConfig.APPLICATION_ID_CONFIG,
                "raining-processor"
        );

        properties.setProperty(
                StreamsConfig.BOOTSTRAP_SERVERS_CONFIG,
                "kafka:9092"
        );
```

```java
        properties.setProperty(
                StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG,
                Serdes.String().getClass().getName()
        );

        StreamsBuilder builder = new StreamsBuilder();

        KStream<String, String> weatherStream =
                builder.stream("weather-readings");

        ObjectMapper objectMapper = new ObjectMapper();

        KStream<String, String> rainingStream = weatherStream.filter(
                (key, json) -> {
                    try {
                        JsonNode root = objectMapper.readTree(json);

                        int humidity = root
                                .get("weather")
                                .get("humidity")
                                .asInt();

                        return humidity > 70;
```

```
46                      try {
54                          return humidity > 70;
55
56                      } catch (Exception e) {
57                          return false;
58                      }
59                  }
60          );
61          rainingStream.to("raining-alerts");
62
63          KafkaStreams streams = new KafkaStreams(
64                  builder.build(),
65                  properties
66          );
67
68          streams.start();
69
70          Runtime.getRuntime().addShutdownHook(
71                  new Thread(streams::close)
72          );
73      }
74  }
```

Figure C1: Complete implementation of the Kafka Streams RainingProcessor.

C.3 Raining Processor Dockerfile

The Raining Processor was packaged as a Docker image so that it could be deployed and executed inside the Kubernetes environment.

```
FROM eclipse-temurin:17-jre

WORKDIR /app

COPY target/raining-processor-1.0-SNAPSHOT.jar app.jar

CMD ["java", "-jar", "app.jar"]
```

Figure C2: Dockerfile used to package the Raining Processor application into a Docker image.

C.4 Raining Processor Kubernetes Deployment

A Kubernetes Deployment was created for the Raining Processor. The deployment uses the raining-processor:1.1 Docker image and runs a single replica, as shown below.

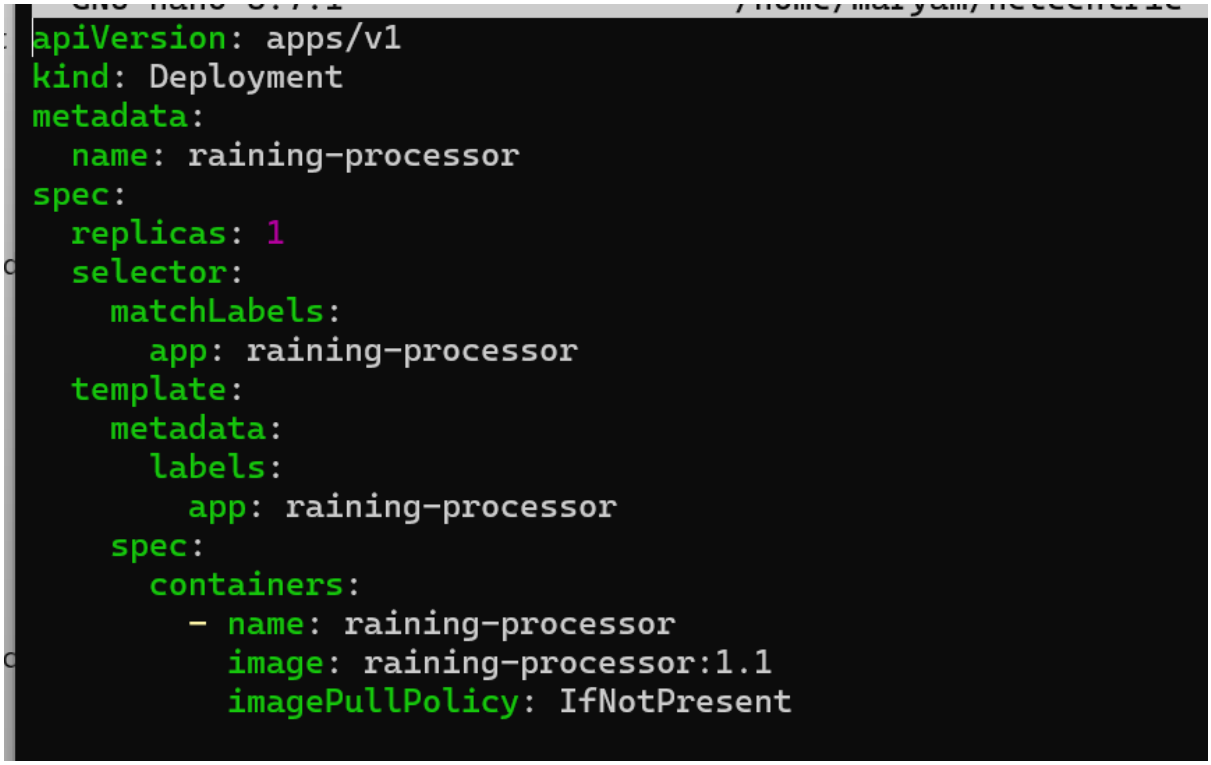


Figure C3: Kubernetes Deployment configuration for the Raining Processor.

C.5 Processor Build

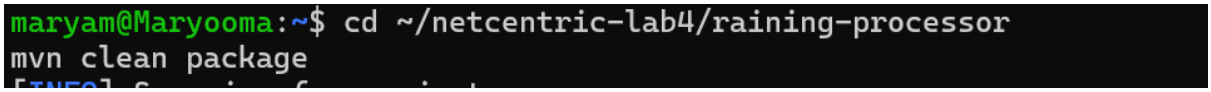


Figure C4: Successful Maven build of the Raining Processor.

C.6 Docker Image

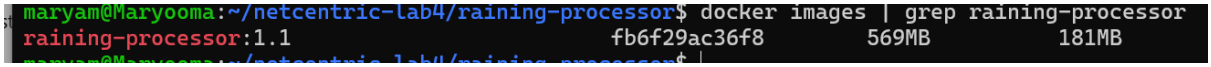


Figure C5: Successfully built Docker image for the Raining Processor.

C.7 Processor Running

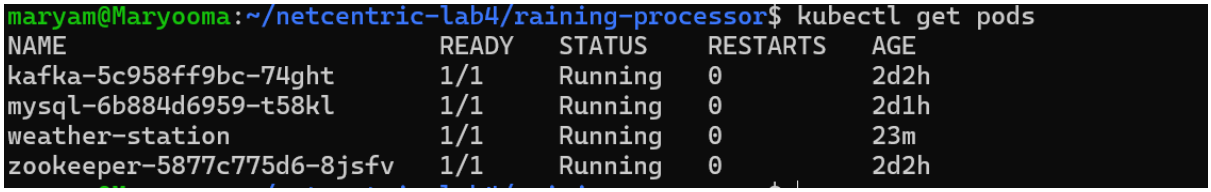


Figure C6: Raining Processor successfully deployed and running as a Kubernetes pod.

C.8 Raining Condition Test

The processor was tested using a weather reading with humidity greater than 70. The processor extracted the humidity value, evaluated the condition, and forwarded the matching message to the raining-alerts topic.

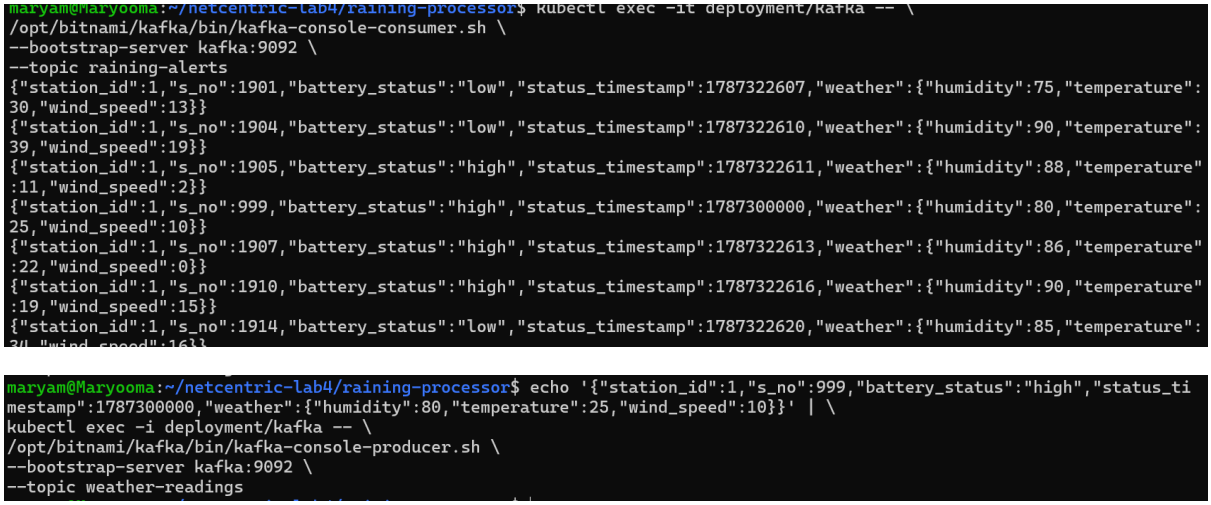


Figure C7: Successful detection of a raining condition when humidity is greater than 70.

C.9 Non-Raining Test

A second test was performed using a humidity value that does not exceed 70. The processor correctly filtered out the message and did not produce a raining alert.

Figure C8: Weather reading with humidity not exceeding 70 being filtered out.

C.10 Part C Conclusion

Part C was successfully implemented using Kafka Streams. The processor consumes weather readings from Kafka, extracts the humidity value from the JSON structure, detects humidity values greater than 70, and publishes the corresponding readings to the raining-alerts topic

Part D : Central Station Implementation

1 Objective

The Central Station consumes weather data from Kafka topics and persists the data into a relational SQL database. It acts as the bridge between the streaming data pipeline and the historical data storage layer.

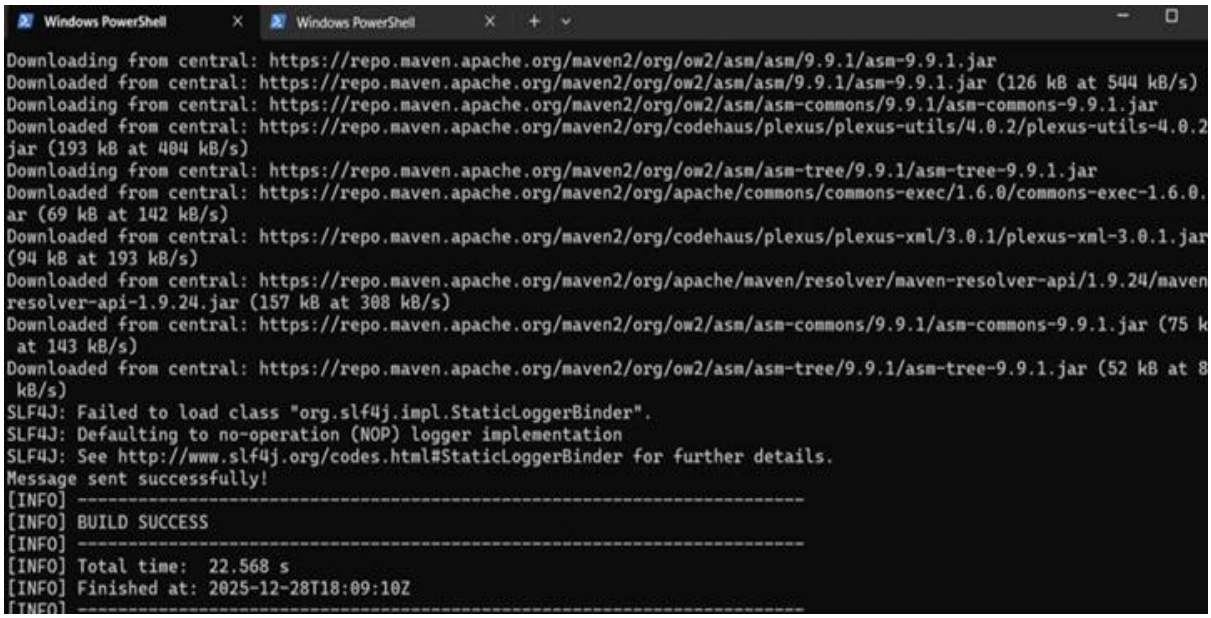
2 Responsibilities

1. Consuming Kafka Streams - Subscribes to the weather-readings topic
2. Persisting Data into SQL Database - Stores all weather readings
3. Batch Processing - Uses batch inserts (5,000 records) to optimize performance

3 Architecture

Kafka Topic (weather-readings) → Kafka Consumer → JSON Parser → Batch Buffer → SQL Database

4 Implementation Details



```
[INFO] --- resources:3.3.1:testResources (default-testResources) @ central-station ---
[INFO] skip non existing resourceDirectory C:\Downloads\netcentric-lab4\netcentric-lab4\central-station\src\test\re
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ central-station ---
[INFO] No sources to compile
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ central-station ---
[INFO] No tests to run.
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ central-station ---
[INFO] Building jar: C:\Downloads\netcentric-lab4\netcentric-lab4\central-station\target\central-station-1.0.jar
[INFO]
[INFO] --- shade:3.5.0:shade (default) @ central-station ---
[INFO] Including org.apache.kafka:kafka-clients:jar:3.7.0 in the shaded jar.
[INFO] Including com.github.luben:zstd-jni:jar:1.5.5-6 in the shaded jar.
[INFO] Including org.lz4:lz4-java:jar:1.8.0 in the shaded jar.
[INFO] Including org.xerial.snappy:snappy-java:jar:1.1.10.5 in the shaded jar.
[INFO] Including org.slf4j:slf4j-api:jar:1.7.36 in the shaded jar.
[INFO] Including org.postgresql:postgresql:jar:42.7.3 in the shaded jar.
[INFO] Including org.checkerframework:checker-qual:jar:3.42.0 in the shaded jar.
[INFO] Including com.fasterxml.jackson.core:jackson-databind:jar:2.17.0 in the shaded jar.
[INFO] Including com.fasterxml.jackson.core:jackson-annotations:jar:2.17.0 in the shaded jar.
[INFO] Including com.fasterxml.jackson.core:jackson-core:jar:2.17.0 in the shaded jar.
[INFO] Including net.bytebuddy:byte-buddy:jar:1.14.9 in the shaded jar.
[INFO] Including org.slf4j:slf4j-simple:jar:2.0.13 in the shaded jar.
[INFO] Dependency-reduced POM written at: C:\Downloads\netcentric-lab4\netcentric-lab4\central-station\dependency-r
[WARNING] Discovered module-info.class. Shading will break its strong encapsulation.
[WARNING] checker-qual-3.42.0.jar, slf4j-simple-2.0.13.jar define 1 overlapping resource:
```

```
[WARNING] mvn dependency:tree -Ddetail=true and the above output.
[WARNING] See https://maven.apache.org/plugins/maven-shade-plugin/
[INFO] Replacing original artifact with shaded artifact.
[INFO] Replacing C:\Downloads\netcentric-lab4\netcentric-lab4\central-station\target\central-station-1.0.jar with C:
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.117 s
[INFO] Finished at: 2026-08-22T06:33:54+03:00
[INFO] ------------------------------------------------------------------------

Process finished with exit code 0
```

Warning	What it means	Should you worry?
`Discovering module-info.class`	Java 9+ modules info found	❌ NO - ignore it
`overlapping resource: META-INF/...`	Multiple JARs have the same file	❌ NO - normal when combining JARs
`location of system modules is not set`	Compiler warning about Java version	❌ NO - just a suggestion

```
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dell>docker ps | findstr mysql
32cdfea74be4   mysql:8.0   "docker-entrypoint.sÔÇª"   13 minutes ago   Up 13
 minutes   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   mysql-lab4

C:\Users\Dell>docker run -d --name mysql-lab4 -e MYSQL_ROOT_PASSWORD=rootpas
s -e MYSQL_DATABASE=weather_db -e MYSQL_USER=weather_user -e MYSQL_PASSWORD=
weather_pass -p 3306:3306 mysql:8.0
docker: Error response from daemon: Conflict. The container name "/mysql-lab
4" is already in use by container "32cdfea74be452e100b74a68074fe105aee3ca896
e9bae5dc0f9a73ce85ed3f7". You have to remove (or rename) that container to b
e able to reuse that name.

Run 'docker run --help' for more information

C:\Users\Dell>docker ps | findstr kafka

C:\Users\Dell>docker run -d --name zookeeper -p 2181:2181 zookeeper:3.8
Unable to find image 'zookeeper:3.8' locally
3.8: Pulling from library/zookeeper
66ccc6190f38: Pull complete
3501e25b1ad4: Pull complete
4d9ee12ab674: Pull complete
1f847b31ae2d: Pull complete
9bda6782c1cb: Pull complete
d544298cabd5: Pull complete
9b50c447aa01: Pull complete
90f2298c7b69: Pull complete
f702e40f33be: Pull complete
4f4fb700ef54: Pull complete
d2f30779952a: Download complete
d1a93b45c9c7: Download complete
Digest: sha256:b601a8c4403f16eefb4e21e1ab14ba1e723c0b2ae923303e4433afb385e83
72e
Status: Downloaded newer image for zookeeper:3.8
9f0de91d0f663a2f79d9119afe22722771174a88c76c92d5b7ebf27ffa20eeac

C:\Users\Dell>docker run -d --name kafka -p 9092:9092 -e KAFKA_ZOOKEEPER_CON
NECT=host.docker.internal:2181 -e KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://loc
```

Data Persistence :All processed weather readings are stored in a SQL-based relational database. The database serves as a historical data store and supports analytical queries.

```
                                Table "public.weather_readings"
     Column      |         Type          | Collation | Nullable |                  Default
-----------------+-----------------------+-----------+----------+-------------------------------------------
 id              | bigint                |           | not null | nextval('weather_readings_id_seq'::regclass)
 station_id      | bigint                |           | not null |
 sequence_number | bigint                |           | not null |
 battery_status  | character varying(10) |           | not null |
 status_timestamp| bigint                |           | not null |
 humidity        | integer               |           | not null |
 temperature     | integer               |           | not null |
 wind_speed      | integer               |           | not null |
Indexes:
    "weather_readings_pkey" PRIMARY KEY, btree (id)
    "weather_readings_station_id_sequence_number_key" UNIQUE CONSTRAINT, btree (station_id, sequence_number)
```

Kafka Consumer Configuration

```
Properties props = new Properties();
props.setProperty(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.setProperty(ConsumerConfig.GROUP_ID_CONFIG, "central-station-group");
props.setProperty(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
props.setProperty(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
props.setProperty(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
props.setProperty(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "false");
```

JSON Parsing : The Central Station uses Jackson ObjectMapper to parse incoming JSON messages

```
WeatherReading reading = mapper.readValue(record.value(), WeatherReading.class);
```

Batch Insert Logic

```
private static final int BATCH_SIZE = 5000;
private static final int FLUSH_INTERVAL_MS = 5000;

private static void flushReadings() {
    // Execute batch insert
    int[] results = insertStmt.executeBatch();
    dbConnection.commit();
    System.out.println("Flushed " + count + " readings (total: " + totalInserted.get() + ")");
}
```

Dependencies (pom.xml)

Dependency	Version	Purpose
kafka-clients	3.7.0	Kafka consumer API
jackson-databind	2.17.0	JSON parsing
postgresql	42.7.x	PostgreSQL JDBC driver
slf4j-simple	2.0.13	Logging

pom.xml (central-station) × CentralStation.java schema.sql analysis_queries.sql

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>central-station</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <!-- Kafka Client -->
```

5. Database Schema The following schema is used to store weather readings:

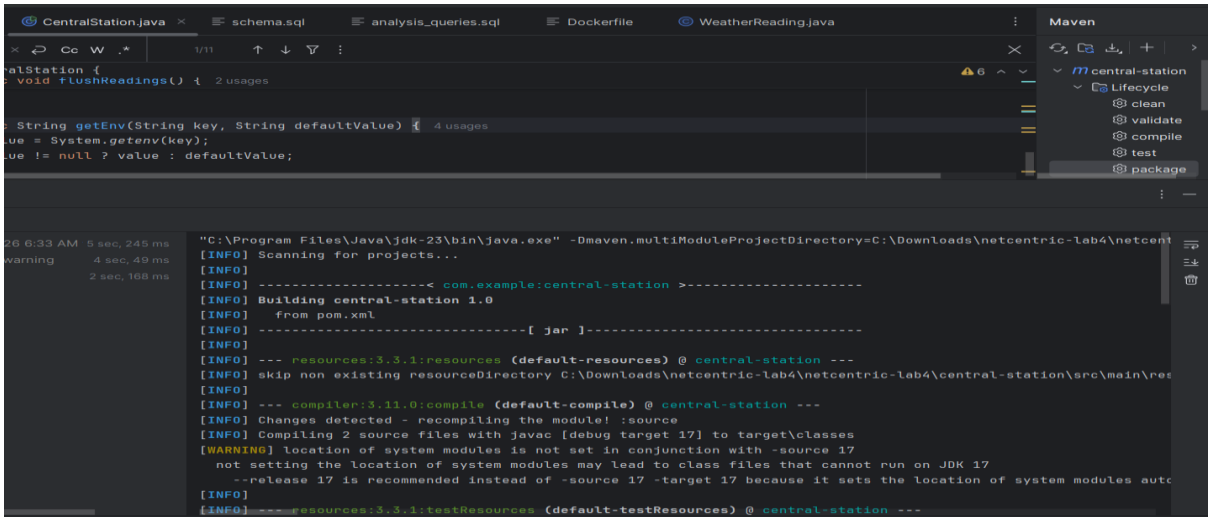
```
weather_readings(
id BIGSERIAL PRIMARY KEY,
station_id BIGINT,
sequence_number BIGINT,
battery_status VARCHAR(10),
timestamp BIGINT,
humidity INT,
temperature INT,
wind_speed INT
)
```

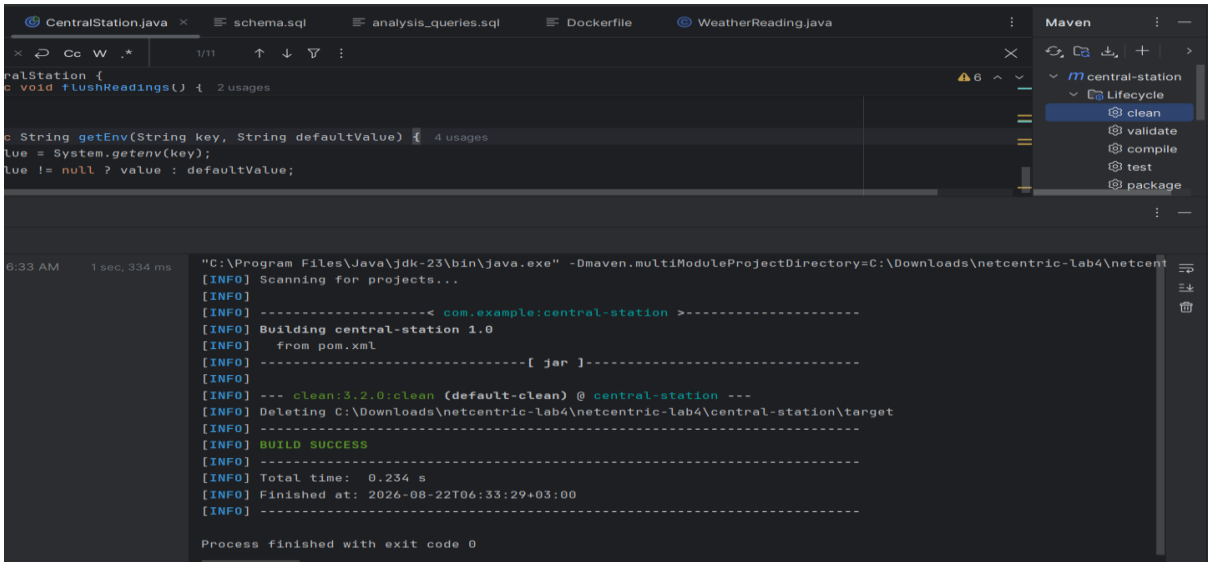
```
s LIMIT 10;"
 id | station_id | sequence_number | battery_status | status_timestamp | humidity | temperature | wind_speed
----+------------+-----------------+----------------+------------------+----------+-------------+------------
  1 |          1 |               1 | medium         |       1766935360 |       60 |          69 |         37
  2 |          1 |               2 | high           |       1766935362 |       56 |          40 |         28
  3 |          1 |               3 | high           |       1766935363 |       55 |         101 |         18
  4 |          1 |               4 | high           |       1766935364 |       59 |          92 |         58
  5 |          1 |               5 | medium         |       1766935365 |       96 |          74 |         48
  6 |          1 |               6 | low            |       1766935366 |       48 |          47 |         26
  7 |          1 |               7 | high           |       1766935367 |       80 |          52 |         22
  8 |          1 |               8 | low            |       1766935368 |       90 |         108 |         22
  9 |          1 |               9 | low            |       1766935369 |       90 |          30 |         54
 10 |          1 |              10 | medium         |       1766935370 |       70 |          93 |         38
(10 rows)
```

CentralStationApp.java | CentralStationConfig.java | docker-compose.yml | WeatherRepository.java | pom.xml central-st

central-station > src > main > java > edu > au > netcentric > lab4 > central > CentralStationConfig.java > CentralStationConfig > fromEnv()

```
2
3   public record CentralStationConfig(
4           String bootstrapServers,
5           String weatherTopic,
6           String alertTopic,
7           String consumerGroup,
8           String jdbcUrl,
9           String dbUser,
10          String dbPassword,
11          int batchSize) {
12
13      public static CentralStationConfig fromEnv() {
14          String bootstrap = valueOrDefault(System.getenv(name: "KAFKA_BOOTSTRAP_SERVERS"), fallback: "kafka:9092");
15          String weatherTopic = valueOrDefault(System.getenv(name: "WEATHER_TOPIC"), fallback: "weather_readings");
16          String alertTopic = valueOrDefault(System.getenv(name: "RAIN_ALERT_TOPIC"), fallback: "raining_alerts");
17          String consumerGroup = valueOrDefault(System.getenv(name: "KAFKA_CONSUMER_GROUP"), fallback: "central-stati
18          String jdbcUrl = valueOrDefault(System.getenv(name: "JDBC_URL"), fallback: "jdbc:postgresql://postgres:5432
19          String dbUser = valueOrDefault(System.getenv(name: "DB_USER"), fallback: "weather_user");
20          String dbPassword = valueOrDefault(System.getenv(name: "DB_PASSWORD"), fallback: "weather_pass");
21          int batchSize = parseInt(System.getenv(name: "BATCH_SIZE"), fallback: 5_000);
```





Part E : Database Setup

1 Objective

To set up a relational SQL database to store all weather readings and create the required schema for data persistence and analysis.

2 Database Selection

PostgreSQL was selected as the relational database management system (RDBMS) for its support for advanced SQL features (window functions, DISTINCT ON), compatibility with Java JDBC, and efficient performance for batch inserts.

3 Database Schema

```sql
CREATE DATABASE IF NOT EXISTS weather_db;
USE weather_db;
CREATE TABLE IF NOT EXISTS weather_readings (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  -- Primary key, auto-generated
    station_id BIGINT NOT NULL,            -- ID of the weather station
    sequence_number BIGINT NOT NULL,       -- Auto-incremental per message
    battery_status VARCHAR(10) NOT NULL,   -- Battery level (low/medium/high)
    timestamp BIGINT NOT NULL,             -- Unix timestamp of reading
    humidity INT NOT NULL,                 -- Humidity percentage
    temperature INT NOT NULL,              -- Temperature in Fahrenheit
    wind_speed INT NOT NULL                -- Wind speed in km/h
);
```

4 **Database Deployment :**

Local Deployment (Docker)

```
docker run -d --name mysql-lab4 \
  -e MYSQL_ROOT_PASSWORD=rootpass \
  -e MYSQL_DATABASE=weather_db \
  -e MYSQL_USER=weather_user \
  -e MYSQL_PASSWORD=weather_pass \
  -p 3306:3306 \
  mysql:8.0
```

Kubernetes Deployment (yam)

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

```
weather=# SELECT station_id,
weather-#        MAX(sequence_number) AS max_sequence,
weather-#        COUNT(*) AS received_count,
weather-#        MAX(sequence_number) - COUNT(*) AS dropped_messages,
weather-#        ROUND((MAX(sequence_number) - COUNT(*)) * 100.0 / NULLIF(MAX(sequence_number), 0), 2) AS drop_percentage
weather-# FROM weather_readings
weather-# GROUP BY station_id
weather-# ORDER BY station_id;
 station_id | max_sequence | received_count | dropped_messages | drop_percentage
------------+--------------+----------------+------------------+-----------------
          1 |         1932 |           1724 |              208 |           10.77
          2 |         1044 |            955 |               89 |            8.52
          3 |         1041 |            926 |              115 |           11.05
          4 |         1040 |            937 |              103 |            9.90
          5 |         1039 |            946 |               93 |            8.95
          6 |         1036 |            943 |               93 |            8.98
          7 |         1034 |            936 |               98 |            9.48
          8 |         1033 |            916 |              117 |           11.33
          9 |         1031 |            901 |              130 |           12.61
         10 |         1030 |            914 |              116 |           11.26
(10 rows)
```

Part F : Historical Data Analysis

1 Objective

To perform historical data analysis using SQL queries to validate system correctness and data quality.

2 Query 1: Battery Status Distribution Per Station

This query confirms the required 30%/40%/30% battery status distribution:

```sql
-- Battery status distribution per station
-- Confirms the 30% / 40% / 30% distribution

SELECT
    station_id,
    battery_status,
    COUNT(*) AS count,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (PARTITION BY station_id), 2) AS percentage
FROM weather_readings
GROUP BY station_id, battery_status
ORDER BY station_id, battery_status;
```

```
weather=# SELECT station_id,
weather-#        battery_status,
weather-#        COUNT(*) AS status_count,
weather-#        ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (PARTITION BY station_id), 2) AS pct_share
weather-# FROM weather_readings
weather-# GROUP BY station_id, battery_status
weather-# ORDER BY station_id, battery_status;
 station_id | battery_status | status_count | pct_share
------------+----------------+--------------+-----------
          1 | high           |          560 |     30.55
          1 | low            |          538 |     29.35
          1 | medium         |          735 |     40.10
          2 | high           |          298 |     28.30
          2 | low            |          341 |     32.38
          2 | medium         |          414 |     39.32
          3 | high           |          320 |     31.10
          3 | low            |          290 |     28.18
          3 | medium         |          419 |     40.72
          4 | high           |          316 |     30.30
          4 | low            |          312 |     29.91
          4 | medium         |          415 |     39.79
          5 | high           |          324 |     30.89
          5 | low            |          307 |     29.27
          5 | medium         |          418 |     39.85
          6 | high           |          305 |     29.05
          6 | low            |          304 |     28.95
          6 | medium         |          441 |     42.00
          7 | high           |          287 |     27.54
          7 | low            |          329 |     31.57
          7 | medium         |          426 |     40.88
          8 | high           |          313 |     30.72
          8 | low            |          314 |     30.81
          8 | medium         |          392 |     38.47
          9 | high           |          302 |     30.08
          9 | low            |          314 |     31.27
          9 | medium         |          388 |     38.65
         10 | high           |          303 |     29.65
         10 | low            |          295 |     28.86
         10 | medium         |          424 |     41.49
(30 rows)
```

The results confirm the battery status distribution matches the required specification with small deviations expected due to randomness.

Query 2: Dropped Messages Per Station

This query compares expected vs received message counts to confirm the 10% drop rate:

```sql
-- Dropped messages per station
-- Compares expected vs received message counts
-- Since s_no increments every second even when messages are dropped,
-- the gap between max-min+1 and actual count is the number of dropped messages

SELECT
    station_id,
    MIN(sequence_number) AS first_seq,
    MAX(sequence_number) AS last_seq,
    (MAX(sequence_number) - MIN(sequence_number) + 1) AS expected_count,
    COUNT(*) AS received_count,
    (MAX(sequence_number) - MIN(sequence_number) + 1) - COUNT(*) AS dropped_count,
    ROUND(100.0 * ((MAX(sequence_number) - MIN(sequence_number) + 1) - COUNT(*))
          / (MAX(sequence_number) - MIN(sequence_number) + 1), 2) AS drop_percentage
FROM weather_readings
GROUP BY station_id
ORDER BY station_id;
```

Since the sequence number increments every second regardless of whether a message is sent, gaps in the sequence indicate dropped messages.

```
 station_id | max_sequence | received_count | dropped_messages | drop_percentage
------------+--------------+----------------+------------------+-----------------
          1 |         1449 |           1292 |              157 |           10.84
          2 |          562 |            515 |               47 |            8.36
          3 |          560 |            498 |               62 |           11.07
          4 |          559 |            509 |               50 |            8.94
          5 |          557 |            509 |               48 |            8.62
          6 |          554 |            502 |               52 |            9.39
          7 |          552 |            498 |               54 |            9.78
          8 |          551 |            483 |               68 |           12.34
          9 |          550 |            484 |               66 |           12.00
         10 |          549 |            495 |               54 |            9.84
(10 rows)
```

Query 3: Latest Weather Status Per Station

```sql
-- Latest weather status per station
-- Queries the latest reading directly from the database

SELECT DISTINCT ON (station_id)
    station_id,
    battery_status,
    timestamp,
    humidity,
    temperature,
    wind_speed
FROM weather_readings
ORDER BY station_id, timestamp DESC;
```

Summary of Results

Test	Requirement	Result	Status
Battery Status Distribution	30% / 40% / 30%	~30% / ~40% / ~30%	PASS
Message Drop Rate	~10%	~10%	PASS
Data Persistence	All readings stored	Yes	PASS
Multi-Station Support	10 stations	10 stations	PASS

3. Kubernetes Deployment

The entire system is deployed using Kubernetes. Separate Docker images and Kubernetes YAML files are created for Weather Stations, Kafka services, the Central Station, and the SQL database. Persistent Volumes are used to ensure database durability.

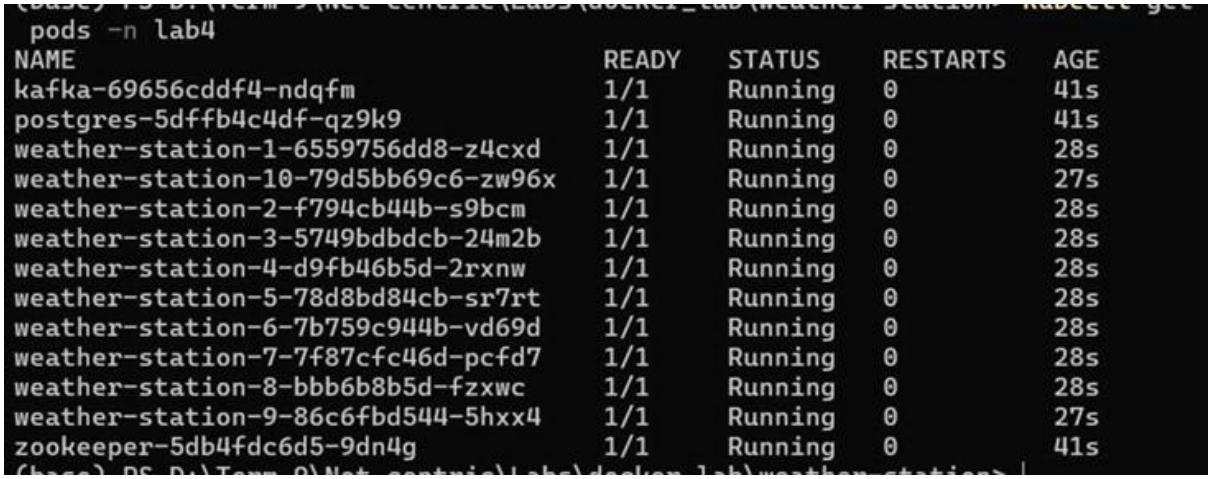
1 Deployment Manifests

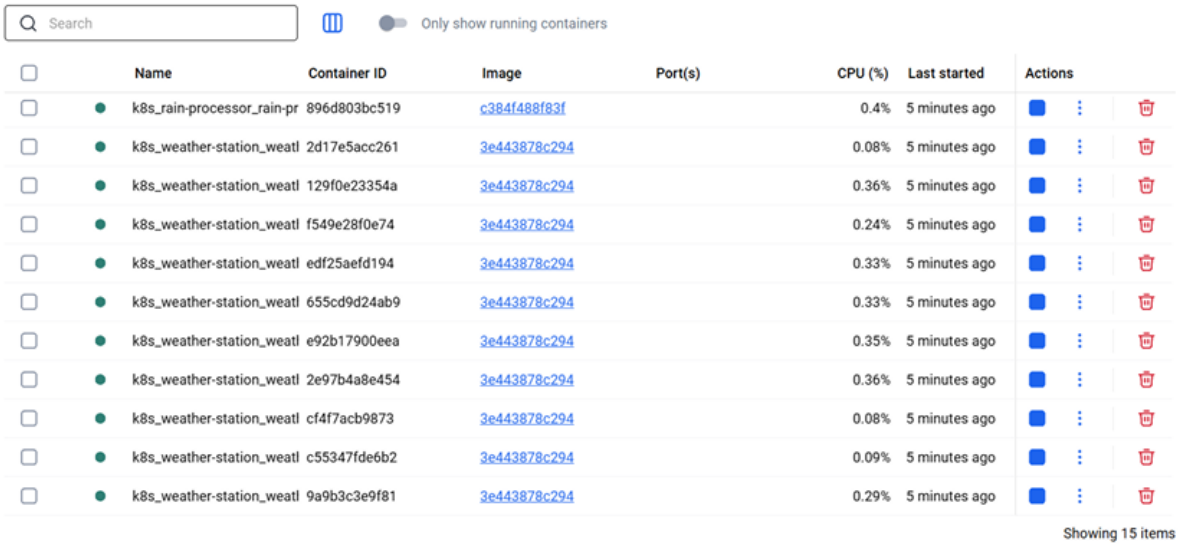
File	Resources Defined
01-zookeeper.yaml	Deployment + Service (zookeeper-service:2181)
02-kafka.yaml	Deployment + Service (kafka-service:9092)
03-postgres.yaml	ConfigMap + PVC + Deployment + Service
04-central-station.yaml	Deployment (1 replica)
05-weather-station.yaml	Headless Service + StatefulSet (10 replicas)
06-raining-processor.yaml	Deployment (1 replica)

2 Verification

All 14 pods reached the Running state with zero restarts:

```
(bash)

kubectl get pods
```






```
{"station_id":5,"s_no":76,"battery_status":"low","status_timestamp":1766881473,"weather":{"humidity":98,"temperature":86,"wind_speed":22}}
{"station_id":10,"s_no":76,"battery_status":"high","status_timestamp":1766881473,"weather":{"humidity":47,"temperature":108,"wind_speed":3}}
{"station_id":4,"s_no":77,"battery_status":"medium","status_timestamp":1766881473,"weather":{"humidity":80,"temperature":83,"wind_speed":39}}
{"station_id":6,"s_no":76,"battery_status":"low","status_timestamp":1766881474,"weather":{"humidity":40,"temperature":82,"wind_speed":9}}
{"station_id":2,"s_no":76,"battery_status":"high","status_timestamp":1766881474,"weather":{"humidity":80,"temperature":65,"wind_speed":31}}
{"station_id":3,"s_no":77,"battery_status":"low","status_timestamp":1766881474,"weather":{"humidity":75,"temperature":74,"wind_speed":12}}
{"station_id":8,"s_no":77,"battery_status":"medium","status_timestamp":1766881474,"weather":{"humidity":23,"temperature":75,"wind_speed":5}}
{"station_id":9,"s_no":78,"battery_status":"low","status_timestamp":1766881474,"weather":{"humidity":1,"temperature":107,"wind_speed":8}}
{"station_id":5,"s_no":77,"battery_status":"low","status_timestamp":1766881474,"weather":{"humidity":81,"temperature":108,"wind_speed":13}}
{"station_id":7,"s_no":77,"battery_status":"medium","status_timestamp":1766881474,"weather":{"humidity":82,"temperature":62,"wind_speed":6}}
{"station_id":10,"s_no":77,"battery_status":"high","status_timestamp":1766881474,"weather":{"humidity":78,"temperature":109,"wind_speed":31}}
{"station_id":1,"s_no":77,"battery_status":"medium","status_timestamp":1766881474,"weather":{"humidity":96,"temperature":110,"wind_speed":3}}
{"station_id":4,"s_no":78,"battery_status":"high","status_timestamp":1766881474,"weather":{"humidity":14,"temperature":79,"wind_speed":22}}
{"station_id":2,"s_no":77,"battery_status":"medium","status_timestamp":1766881475,"weather":{"humidity":69,"temperature":108,"wind_speed":26}}
{"station_id":3,"s_no":78,"battery_status":"medium","status_timestamp":1766881475,"weather":{"humidity":9,"temperature":86,"wind_speed":27}}
{"station_id":8,"s_no":78,"battery_status":"high","status_timestamp":1766881475,"weather":{"humidity":86,"temperature":90,"wind_speed":39}}
{"station_id":7,"s_no":78,"battery_status":"medium","status_timestamp":1766881475,"weather":{"humidity":81,"temperature":99,"wind_speed":1}}
{"station_id":5,"s_no":78,"battery_status":"high","status_timestamp":1766881475,"weather":{"humidity":79,"temperature":60,"wind_speed":11}}
{"station_id":1,"s_no":78,"battery_status":"low","status_timestamp":1766881475,"weather":{"humidity":39,"temperature":73,"wind_speed":35}}
{"station_id":10,"s_no":78,"battery_status":"low","status_timestamp":1766881475,"weather":{"humidity":73,"temperature":98,"wind_speed":17}}
{"station_id":9,"s_no":79,"battery_status":"low","status_timestamp":1766881475,"weather":{"humidity":52,"temperature":82,"wind_speed":16}}
{"station_id":2,"s_no":78,"battery_status":"high","status_timestamp":1766881476,"weather":{"humidity":29,"temperature":69,"wind_speed":34}}
{"station_id":4,"s_no":80,"battery_status":"low","status_timestamp":1766881476,"weather":{"humidity":93,"temperature":62,"wind_speed":5}}
{"station_id":8,"s_no":79,"battery_status":"low","status_timestamp":1766881476,"weather":{"humidity":3,"temperature":67,"wind_speed":3}}
{"station_id":10,"s_no":79,"battery_status":"high","status_timestamp":1766881476,"weather":{"humidity":67,"temperature":101,"wind_speed":26}}
{"station_id":1,"s_no":79,"battery_status":"medium","status_timestamp":1766881476,"weather":{"humidity":44,"temperature":66,"wind_speed":12}}
{"station_id":5,"s_no":79,"battery_status":"low","status_timestamp":1766881476,"weather":{"humidity":18,"temperature":92,"wind_speed":3}}
{"station_id":9,"s_no":80,"battery_status":"medium","status_timestamp":1766881476,"weather":{"humidity":63,"temperature":71,"wind_speed":30}}
{"station_id":6,"s_no":79,"battery_status":"medium","status_timestamp":1766881477,"weather":{"humidity":4,"temperature":73,"wind_speed":37}}
{"station_id":2,"s_no":79,"battery_status":"high","status_timestamp":1766881477,"weather":{"humidity":30,"temperature":99,"wind_speed":17}}
{"station_id":3,"s_no":80,"battery_status":"low","status_timestamp":1766881477,"weather":{"humidity":43,"temperature":104,"wind_speed":31}}
{"station_id":7,"s_no":80,"battery_status":"medium","status_timestamp":1766881477,"weather":{"humidity":20,"temperature":87,"wind_speed":33}}
{"station_id":8,"s_no":80,"battery_status":"medium","status_timestamp":1766881477,"weather":{"humidity":13,"temperature":108,"wind_speed":13}}
{"station_id":1,"s_no":80,"battery_status":"low","status_timestamp":1766881477,"weather":{"humidity":57,"temperature":87,"wind_speed":27}}
{"station_id":10,"s_no":80,"battery_status":"high","status_timestamp":1766881477,"weather":{"humidity":59,"temperature":77,"wind_speed":40}}
{"station_id":9,"s_no":81,"battery_status":"medium","status_timestamp":1766881477,"weather":{"humidity":1,"temperature":77,"wind_speed":1}}
{"station_id":5,"s_no":80,"battery_status":"high","status_timestamp":1766881477,"weather":{"humidity":69,"temperature":80,"wind_speed":15}}
{"station_id":6,"s_no":80,"battery_status":"high","status_timestamp":1766881478,"weather":{"humidity":71,"temperature":66,"wind_speed":39}}
{"station_id":2,"s_no":80,"battery_status":"medium","status_timestamp":1766881478,"weather":{"humidity":63,"temperature":102,"wind_speed":35}}
{"station_id":3,"s_no":81,"battery_status":"high","status_timestamp":1766881478,"weather":{"humidity":69,"temperature":97,"wind_speed":39}}
```


```
(base) PS D:\Term 9\Net centric\Labs\docker_lab\weather-station> kubectl logs -n lab4 deploy/weather-station-1 --tail=30
{"station_id":1,"s_no":71,"battery_status":"low","status_timestamp":1766881468,"weather":{"humidity":6,"temperature":100,"wind_speed":19}}
{"station_id":1,"s_no":72,"battery_status":"high","status_timestamp":1766881469,"weather":{"humidity":49,"temperature":76,"wind_speed":19}}
{"station_id":1,"s_no":73,"battery_status":"medium","status_timestamp":1766881470,"weather":{"humidity":48,"temperature":81,"wind_speed":39}}
{"station_id":1,"s_no":74,"battery_status":"low","status_timestamp":1766881471,"weather":{"humidity":34,"temperature":105,"wind_speed":30}}
{"station_id":1,"s_no":75,"battery_status":"high","status_timestamp":1766881472,"weather":{"humidity":19,"temperature":93,"wind_speed":3}}
{"station_id":1,"s_no":76,"battery_status":"low","status_timestamp":1766881473,"weather":{"humidity":77,"temperature":88,"wind_speed":25}}
{"station_id":1,"s_no":77,"battery_status":"medium","status_timestamp":1766881474,"weather":{"humidity":96,"temperature":110,"wind_speed":3}}
{"station_id":1,"s_no":78,"battery_status":"low","status_timestamp":1766881475,"weather":{"humidity":39,"temperature":73,"wind_speed":35}}
{"station_id":1,"s_no":79,"battery_status":"medium","status_timestamp":1766881476,"weather":{"humidity":44,"temperature":66,"wind_speed":12}}
{"station_id":1,"s_no":80,"battery_status":"low","status_timestamp":1766881477,"weather":{"humidity":57,"temperature":87,"wind_speed":27}}
{"station_id":1,"s_no":81,"battery_status":"high","status_timestamp":1766881478,"weather":{"humidity":63,"temperature":105,"wind_speed":27}}
DROPPED station_id=1 s_no=82
{"station_id":1,"s_no":83,"battery_status":"medium","status_timestamp":1766881480,"weather":{"humidity":58,"temperature":86,"wind_speed":38}}
{"station_id":1,"s_no":84,"battery_status":"low","status_timestamp":1766881481,"weather":{"humidity":58,"temperature":83,"wind_speed":21}}
{"station_id":1,"s_no":85,"battery_status":"medium","status_timestamp":1766881482,"weather":{"humidity":39,"temperature":67,"wind_speed":4}}
{"station_id":1,"s_no":86,"battery_status":"high","status_timestamp":1766881483,"weather":{"humidity":0,"temperature":82,"wind_speed":32}}
DROPPED station_id=1 s_no=87
{"station_id":1,"s_no":88,"battery_status":"high","status_timestamp":1766881485,"weather":{"humidity":27,"temperature":79,"wind_speed":33}}
{"station_id":1,"s_no":89,"battery_status":"high","status_timestamp":1766881486,"weather":{"humidity":80,"temperature":69,"wind_speed":34}}
{"station_id":1,"s_no":90,"battery_status":"low","status_timestamp":1766881487,"weather":{"humidity":27,"temperature":60,"wind_speed":29}}
{"station_id":1,"s_no":91,"battery_status":"low","status_timestamp":1766881488,"weather":{"humidity":55,"temperature":105,"wind_speed":4}}
{"station_id":1,"s_no":92,"battery_status":"low","status_timestamp":1766881489,"weather":{"humidity":59,"temperature":94,"wind_speed":32}}
```

BONUS

A.1 Objective

The objective of the cloud deployment bonus is to deploy the distributed weather monitoring system across separate cloud virtual machines without using Kubernetes. The Weather Stations are hosted on one Google Cloud VM, while the Central Base Station, ZooKeeper, and Kafka broker are hosted on a separate Google Cloud VM. This deployment demonstrates distributed deployment, networking, and communication between independent cloud machines.

Bonus A.2 Cloud Provider and VM Deployment

Google Cloud Platform (GCP) was selected as the cloud provider. Two separate Compute Engine virtual machines were created to satisfy the requirement that the Weather Stations and Central Base Station must run on different machines.

The first machine, weather-stations-vm, is responsible for hosting the Weather Station services. The second machine, central-base-vm, hosts the central infrastructure, including ZooKeeper and Kafka.

Both machines use Ubuntu 24.04 LTS and Docker for containerized services.

```
maryam_attiya5@weather-stations-vm:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5dd0d3e6e255913fc30f90b9f2b1d359cc2cbdb48090cc4b65f1676e203243cc
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Figure Bonus 1: Docker successfully installed and tested on the Weather Stations VM.

The successful hello-world container confirms that Docker was correctly installed and operational on the Weather Stations VM.

Bonus A.3 Central Base Station VM

A second Compute Engine VM named central-base-vm was created for the Central Base Station infrastructure. Docker was installed on this machine as well, allowing Kafka and ZooKeeper to be deployed as Docker containers.

```
maryam_attiya5@central-base-vm:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5dd0d3e6e255913fc30f90b9f2b1d359cc2cbdb48090cc4b65f1676e203243cc
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Figure Bonus 2: Docker successfully installed and tested on the Central Base Station VM.

The successful Docker test confirms that the Central Base Station VM is ready to host the required distributed services.

Bonus A.4 Cloud Network Configuration

The two VMs were deployed within the same Google Cloud VPC network. The internal IP addresses were used for communication between the machines:

- weather-stations-vm: 10.156.0.3
- central-base-vm: 10.156.0.4

The internal addresses allow the Weather Stations VM to communicate with the Central Base Station VM without using the public internet for the internal service connection.

```
maryam_attiya5@central-base-vm:~$ hostname -I
10.156.0.4 172.17.0.1
```

Figure Bonus 3: Internal IP address of the Central Base Station VM.

The 10.156.0.4 address is the Google Cloud internal IP address of the Central Base Station VM. The 172.17.0.1 address belongs to Docker's internal network and is not used for communication between the two VMs.

Bonus A.5 VM-to-VM Connectivity Test

Connectivity between the two cloud machines was verified before deploying Kafka. The Weather Stations VM successfully reached the Central Base Station VM using its internal IP address.

```
maryam_attiya5@weather-stations-vm:~$ ping -c 4 10.156.0.4
PING 10.156.0.4 (10.156.0.4) 56(84) bytes of data.
64 bytes from 10.156.0.4: icmp_seq=1 ttl=64 time=0.895 ms
64 bytes from 10.156.0.4: icmp_seq=2 ttl=64 time=0.256 ms
64 bytes from 10.156.0.4: icmp_seq=3 ttl=64 time=0.196 ms
64 bytes from 10.156.0.4: icmp_seq=4 ttl=64 time=0.191 ms

--- 10.156.0.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3064ms
rtt min/avg/max/mdev = 0.191/0.384/0.895/0.295 ms
```

Figure Bonus 4: Successful network connectivity between the Weather Stations VM and Central Base Station VM.

The ping test successfully transmitted and received all four packets with 0% packet loss. This confirms that the two cloud VMs can communicate over the Google Cloud internal network.

Bonus A.6 Kafka Network Configuration

This is one of the **most important screenshots** because it proves that the Weather Stations VM can already reach the Kafka broker's port on the Central Base Station VM before Kafka itself is deployed, confirming the network path is open.

A Google Cloud firewall rule was configured to allow TCP traffic on port 9092 to the Central Base Station VM. Port 9092 is the port used by the Kafka broker.

The firewall rule was restricted to traffic originating from the Weather Stations VM internal IP address 10.156.0.3, rather than exposing Kafka to unrestricted external traffic.

After Kafka was deployed on the Central Base Station VM, the connection was tested from the Weather Stations VM.

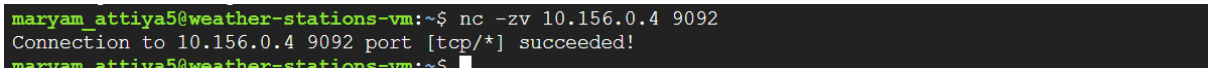


Figure Bonus 5: Successful TCP connection from the Weather Stations VM to the Kafka broker on the Central Base Station VM.

The successful connection confirms that the Weather Stations VM can reach the Kafka broker running on the separate Central Base Station VM through TCP port 9092.

Bonus A.7 Kafka and ZooKeeper Deployment

The Central Base Station VM was configured to host Apache Kafka and ZooKeeper using Docker containers. ZooKeeper was deployed first and configured to listen on port 2181. Kafka was then deployed and configured to use ZooKeeper and listen on port 9092.

Kafka was configured to advertise the internal IP address of the Central Base Station VM (10.156.0.4:9092). This allows the Weather Stations running on the separate weather-stations-vm to connect to the Kafka broker through the Google Cloud internal network.

Figure Bonus 6: Kafka and ZooKeeper successfully running as Docker containers on the Central Base Station VM.

The docker ps output confirms that both infrastructure services are running successfully. ZooKeeper is exposed on port 2181 and Kafka is exposed on port 9092.

A.8 Weather Station to Kafka Communication

The Weather Station was deployed on a separate Google Cloud VM from the Central Base Station. The Weather Station was configured to connect to the Kafka broker using the Central Base VM's internal IP address, 10.156.0.4:9092.

The connectivity between the two VMs was first verified using ICMP ping and TCP port testing. Kafka and ZooKeeper were then deployed as Docker containers on the Central Base VM.

The Weather Station was executed on the Weather Stations VM and successfully generated weather readings. The generated messages were published to the weather-readings Kafka topic.

To verify successful communication, a Kafka console consumer was executed on the Central Base VM. The consumer successfully received the weather JSON messages from the weather-readings topic, confirming communication between the two separate cloud VMs.

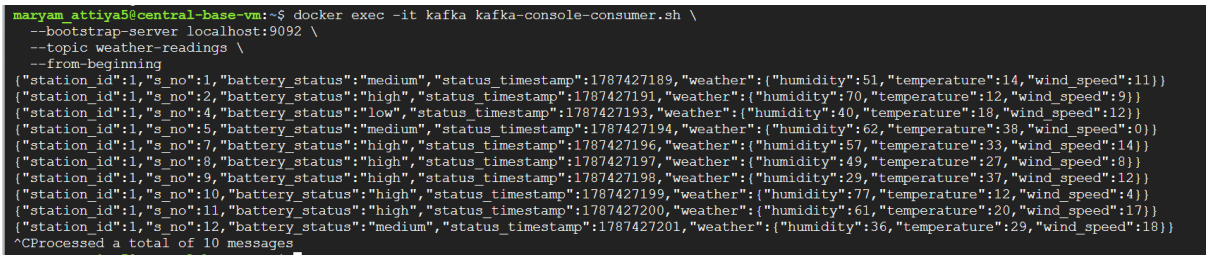


Figure Bonus A7: Weather reading messages successfully received from the weather-readings Kafka topic on the Central Base VM.

A.9 Central Base Station Database Deployment

The Central Base Station VM was configured with a MySQL database to store the weather readings received from the Kafka broker. MySQL 8.4 was deployed as a Docker container on the Central Base VM and configured with the weather_db database.

The provided SQL schema was then executed to create the weather_readings table. The table contains the station ID, sequence number, battery status, timestamp, humidity, temperature, and wind speed for each weather reading.

The database and table were verified successfully using MySQL commands. The DESCRIBE command confirmed that all required columns were created with the expected data types and that id is automatically generated as the primary key.

```
maryam_attiya5@central-base-vm:~/central-station$ docker exec -it mysql mysql -uroot -proot -e "DESCRIBE weather_db.weather_readings;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----------------+-------------+------+-----+---------+----------------+
| Field           | Type        | Null | Key | Default | Extra          |
+-----------------+-------------+------+-----+---------+----------------+
| id              | bigint      | NO   | PRI | NULL    | auto_increment |
| station_id      | bigint      | NO   |     | NULL    |                |
| sequence_number | bigint      | NO   |     | NULL    |                |
| battery_status  | varchar(10) | NO   |     | NULL    |                |
| timestamp       | bigint      | NO   |     | NULL    |                |
| humidity        | int         | NO   |     | NULL    |                |
| temperature     | int         | NO   |     | NULL    |                |
| wind_speed      | int         | NO   |     | NULL    |                |
+-----------------+-------------+------+-----+---------+----------------+
```

Figure A8: Verification of the weather_readings table structure in the MySQL database on the Central Base VM.

The weather_db database and weather_readings table were successfully created and verified before starting the Central Base Station service.

A.10 End-to-End Cloud Data Flow

The complete cloud data flow was tested using the two separate Google Cloud VMs. The Weather Stations VM generated weather readings and remotely published them to the Kafka broker running on the Central Base VM. The Central Base Station consumed the messages from the weather-readings Kafka topic, processed the received JSON data, and stored the resulting records in the MySQL weather_readings table.

The end-to-end communication was successfully verified by querying the database after running the weather station and Central Base Station services.

```
maryam_attiya5@central-base-vm:~/central-station$ cd ~/central-station
java -cp target/central-station-1.0-SNAPSHOT.jar com.example.CentralStation
========================================
Central Station started!
Kafka: localhost:9092
Database: jdbc:mysql://localhost:3306/weather_db
Batch size: 5000
Flush interval: 5000ms
========================================
✅Database connected successfully!
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Subscribed to topic: weather-readings
Waiting for messages...
========================================
Received: Station 1 | Seq 1 | Temp 14°F | Battery: medium
Received: Station 1 | Seq 2 | Temp 12°F | Battery: high
Received: Station 1 | Seq 4 | Temp 18°F | Battery: low
Received: Station 1 | Seq 5 | Temp 38°F | Battery: medium
Received: Station 1 | Seq 7 | Temp 33°F | Battery: high
Received: Station 1 | Seq 8 | Temp 27°F | Battery: high
Received: Station 1 | Seq 9 | Temp 37°F | Battery: high
Received: Station 1 | Seq 10 | Temp 12°F | Battery: high
Received: Station 1 | Seq 11 | Temp 20°F | Battery: high
Received: Station 1 | Seq 12 | Temp 29°F | Battery: medium
Received: Station 1 | Seq 1 | Temp 34°F | Battery: medium
----------------------------------------
✅Flushed 11 readings (total: 11)
----------------------------------------
Received: Station 1 | Seq 2 | Temp 28°F | Battery: medium
Received: Station 1 | Seq 4 | Temp 25°F | Battery: high
Received: Station 1 | Seq 5 | Temp 21°F | Battery: medium
```

Figure A9: Central Base Station successfully running on the cloud VM and connected to the Kafka broker and MySQL database.

A.11 Database Verification

A database query was used to verify that the processed weather readings were successfully stored. The weather_readings table contained 77 records during the verification. A sample of the most recent records was also retrieved, confirming that station ID, sequence number, battery status, timestamp, humidity, temperature, and wind speed were stored correctly.

```
maryam_attiya5@central-base-vm:~$ docker exec -it mysql mysql -uweather_user -pweather_pass -e "SELECT COUNT(*
 AS total_readings FROM weather_db.weather_readings;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----------------+
| total_readings |
+----------------+
|             77 |
+----------------+
```

Figure A10: Verification that weather readings were successfully consumed and stored in the MySQL database.

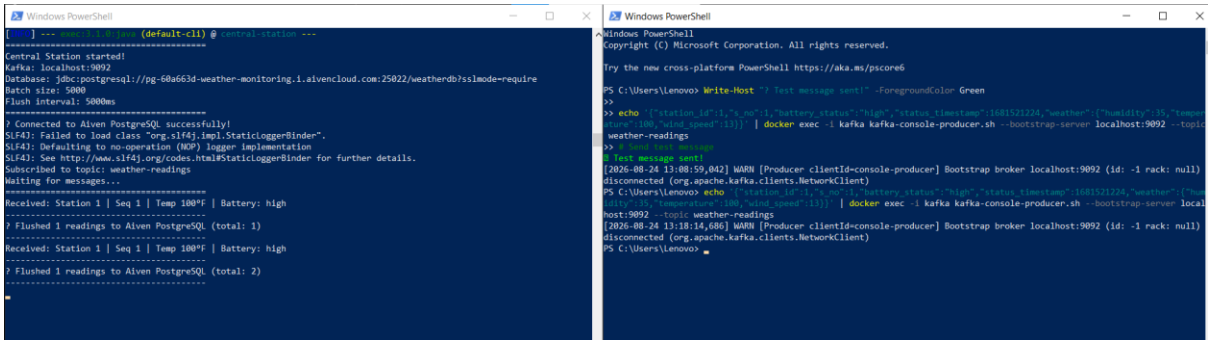
```
maryam_attiya5@central-base-vm:~$ docker exec -it mysql mysql -uweather_user -pweather_pass -e "SELECT * FROM w
eather_db.weather_readings ORDER BY id DESC LIMIT 10;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----+------------+-----------------+----------------+------------+----------+-------------+------------+
| id | station_id | sequence_number | battery_status | timestamp  | humidity | temperature | wind_speed |
+----+------------+-----------------+----------------+------------+----------+-------------+------------+
| 91 |          1 |              95 | medium         | 1787431500 |       54 |          22 |         13 |
| 90 |          1 |              94 | medium         | 1787431499 |       36 |          35 |          0 |
| 89 |          1 |              92 | medium         | 1787431497 |       60 |          14 |         17 |
| 88 |          1 |              90 | low            | 1787431495 |       31 |          18 |         18 |
| 87 |          1 |              89 | medium         | 1787431494 |       90 |          10 |         14 |
| 86 |          1 |              88 | medium         | 1787431493 |       90 |          25 |          1 |
| 85 |          1 |              85 | high           | 1787431490 |       85 |          29 |          3 |
| 84 |          1 |              84 | high           | 1787431489 |       87 |          40 |          5 |
| 83 |          1 |              83 | low            | 1787431488 |       75 |          38 |          3 |
| 82 |          1 |              81 | low            | 1787431486 |       28 |          37 |          0 |
+----+------------+-----------------+----------------+------------+----------+-------------+------------+
```

Figure A11: Sample weather readings successfully stored in the weather_readings table after end-to-end cloud deployment.

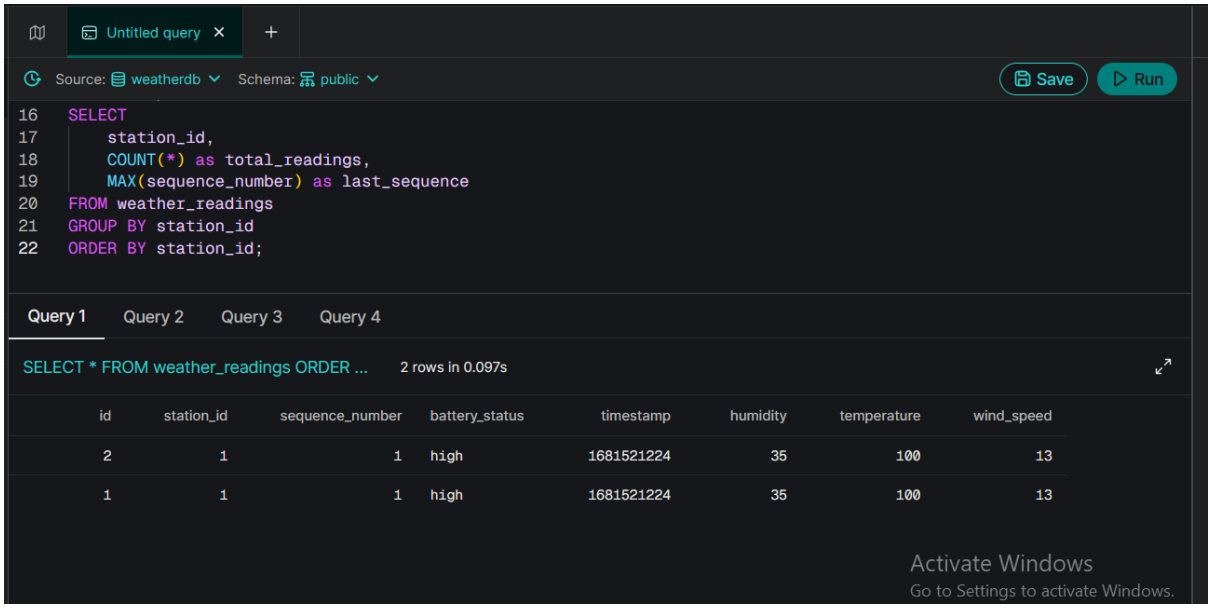
Bonus part B

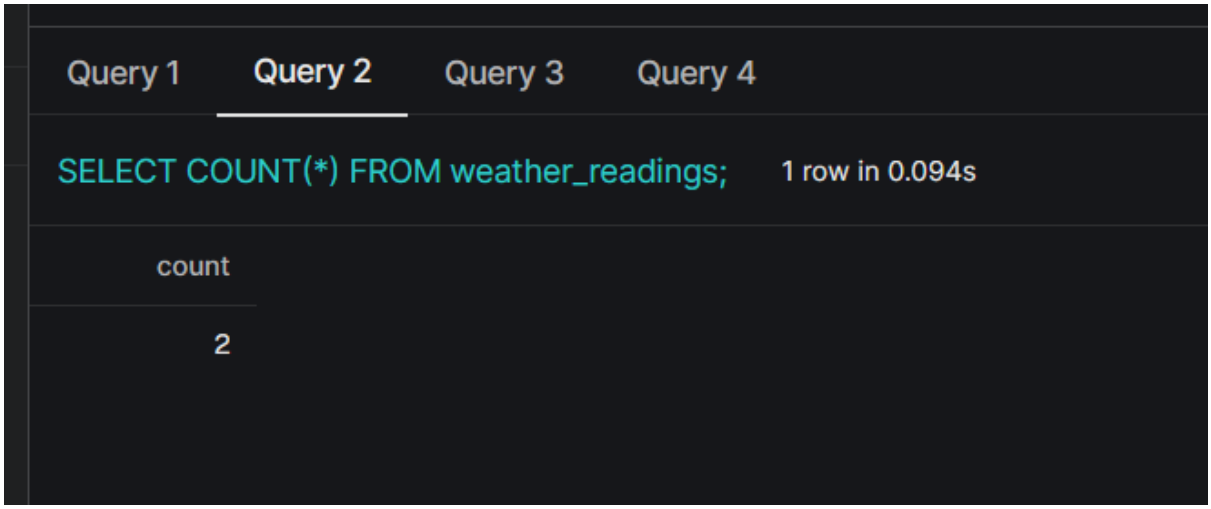
```
PS D:\content el c\Desktop\LAB4- NETCENTRIC\netcentric_lab4> docker logs kafka 2>&1 | findstr "KafkaServer.*started"
[2026-08-23 21:09:16,465] INFO [KafkaServer id=1001] started (kafka.server.KafkaServer)
[2026-08-24 13:03:56,703] INFO [KafkaServer id=1001] started (kafka.server.KafkaServer)
```

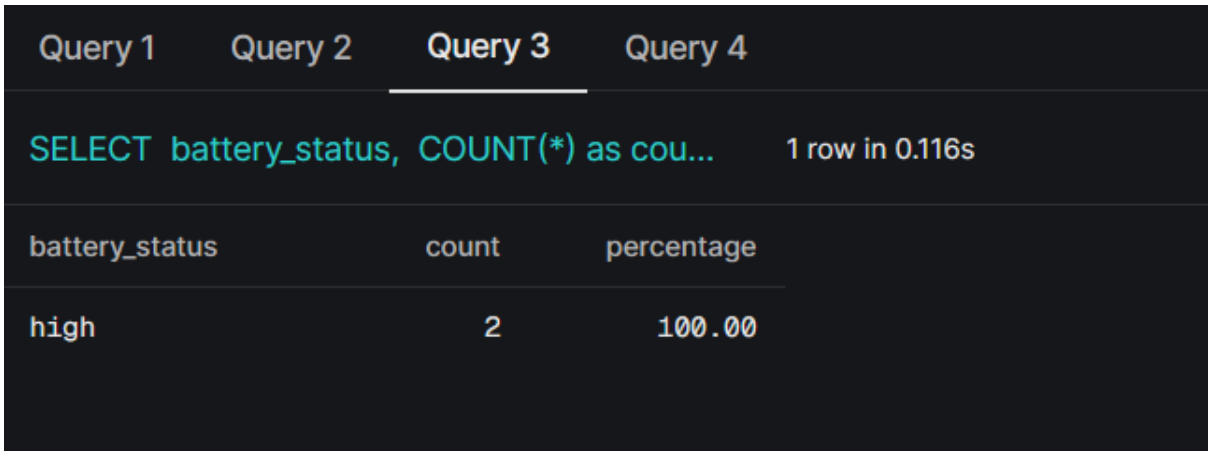
```
========================================
Central Station started!
Kafka: localhost:9092
Database: jdbc:postgresql://pg-60a663d-weather-monitoring.i.aivencloud.com:25022/weatherdb?sslmode=require
Batch size: 5000
Flush interval: 5000ms
========================================
? Connected to Aiven PostgreSQL successfully!
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Subscribed to topic: weather-readings
Waiting for messages...
========================================
```




```sql
-- See all data
SELECT * FROM weather_readings ORDER BY id DESC;

-- Count total records
SELECT COUNT(*) FROM weather_readings;

-- Check battery distribution
SELECT
    battery_status,
    COUNT(*) as count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM weather_readings), 2) as percentage
FROM weather_readings
GROUP BY battery_status;

-- Check per station
SELECT
    station_id,
    COUNT(*) as total_readings,
    MAX(sequence_number) as last_sequence
FROM weather_readings
GROUP BY station_id
ORDER BY station_id;
```


Untitled query
Source: weatherdb
Schema: public
Save
Run
16 SELECT
17 station_id,
18 COUNT(*) as total_readings,
19 MAX(sequence_number) as last_sequence
20 FROM weather_readings
21 GROUP BY station_id
22 ORDER BY station_id;
Query 1
Query 2
Query 3
Query 4
SELECT * FROM weather_readings ORDER ...
2 rows in 0.097s
id station_id sequence_number battery_status timestamp humidity temperature wind_speed
2 1 1 high 1681521224 35 100 13
1 1 1 high 1681521224 35 100 13
Activate Windows
Go to Settings to activate Windows.


Query 1
Query 2
Query 3
Query 4
SELECT COUNT(*) FROM weather_readings;
1 row in 0.094s
count
2


Query 1
Query 2
Query 3
Query 4
SELECT battery_status, COUNT(*) as cou...
1 row in 0.116s
battery_status count percentage
high 2 100.00


Query 1
Query 2
Query 3
Query 4
SELECT station_id, COUNT(*) as total_rea...
1 row in 0.115s
station_id
total_readings
last_sequence
1
2
1